MedAssist AI – Final Project Report

Project Title

AI-Based Medical Symptom Analysis & Disease Prediction System

Prepared by: Adithya Menon
Technology: Python, Machine Learning, CatBoost, FastAPI, Next.js, REST API, Vercel, Render

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Project Overview	3
Problem Statement	3
Objectives	3-2
Machine Learning Model Development	4
Model Evaluation and selection	4-5
Disese Prediction function	5
Application Integration	6
Deployment	6-8
My Contribution	8
Conclusion	9-10

1. Project Overview

MedAssist AI is an AI-based medical symptom analysis and disease prediction system designed to provide users with an initial prediction of a possible disease based on selected symptoms.

The project combines **machine learning, backend API development, frontend development, and cloud deployment** into a single application.

The system allows a user to enter or select symptoms through the frontend. These symptoms are converted into the feature representation expected by the trained machine learning model. The backend then processes the input and sends it to the trained CatBoost classifier.

The predicted disease is returned to the frontend along with additional disease-related information such as:

- Disease description
- Precautions
- Medicines/information
- Diet recommendations
- Lifestyle recommendations
- Recommended specialist
- Severity

The system is intended as an **educational and preliminary symptom-analysis application and not as a replacement for professional medical diagnosis.**

2. Problem Statement

Traditional disease-information systems often require users to search manually through large amounts of medical information.

The objective of MedAssist AI is to provide an interactive system where users can submit symptoms and receive a machine-learning-based prediction of a possible disease along with relevant supporting information.

The project therefore focuses on building a complete pipeline:

User Symptoms → Feature Processing → ML Model → Disease Prediction → Disease Information → User Interface

3. Objectives

The major objectives of the project were:

1. Develop a disease prediction model using a large symptom-disease dataset.
2. Perform exploratory data analysis and preprocessing.

3. Train and compare multiple machine learning models.
4. Select an appropriate final prediction model.
5. Save and integrate the trained model into the application.
6. Develop a symptom-based prediction function.
7. Connect disease predictions with additional disease information.
8. Integrate the prediction model with a backend API.
9. Connect the backend with the frontend application.
10. Deploy the complete application for real-world demonstration.

4.Machine Learning Model Development

Multiple machine learning algorithms were trained and evaluated to determine which approach performed best for the disease classification problem.

The models evaluated included:

- Naive Bayes
- Logistic Regression
- K-Nearest Neighbors
- Random Forest
- Extra Trees
- Decision Tree
- CatBoost

Additional models/approaches were also explored during development, including boosting and deep-learning-based approaches.

The objective was not simply to train one model, but to **compare different approaches and select a model based on evaluation performance.**

5. Model Evaluation and Selection

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

My evaluated models produced the following results:

Model	Accuracy	Precision	Recall	F1 Score
Naive Bayes	86.60%	89.16%	86.60%	86.89%
Logistic Regression	86.25%	86.96%	86.25%	86.34%
KNN	82.82%	83.25%	82.82%	82.88%
Random Forest	75.90%	82.06%	75.90%	76.52%
Extra Trees	74.03%	81.34%	74.03%	75.06%
Decision Tree	13.72%	25.64%	13.72%	16.80%

The team member's CatBoost model achieved approximately:

Accuracy: 86.30%

Macro F1: 86.58%

Although Naive Bayes achieved slightly higher accuracy in my individual experiments, **CatBoost was selected as the final model for application integration based on the team's overall model evaluation and integration decision.**

This allowed the project to maintain a single final prediction model for the deployed application.

6. Disease Prediction Function

After selecting the final model, the next step was to create a reusable prediction function.

The prediction pipeline performs the following operations:

1. Accept symptoms from the user.
2. Create a feature vector containing all 377 model features.
3. Set the selected symptoms to 1.
4. Keep all other symptoms as 0.
5. Pass the feature vector to the trained CatBoost model.
6. Obtain the predicted class.
7. Convert the numerical class back into the disease name using the label encoder.

7. Disease Information Integration

After obtaining the predicted disease, a separate disease-information dataset was integrated with the prediction system.

The dataset contains information such as:

Field	Purpose
Disease	Disease name
Description	General description
Precautions	General precautionary information
Medicines	Medication-related information
Diet Recommendations	General dietary guidance
Workout/Lifestyle	General lifestyle guidance
Specialist	Relevant medical specialist
Severity	General severity category

8. Application Integration

The trained CatBoost model was integrated with the FastAPI backend through a prediction API.

The frontend communicates with the backend using REST API requests.

The overall system flow is:

User → Next.js Frontend → FastAPI Backend → Feature Processing → CatBoost Model → Disease Information → Frontend Result

The integrated application was tested using sample symptom combinations to verify that the model could receive symptoms, generate predictions, and return the corresponding disease information.

9.Deployment

For final deployment, separate platforms were used for the frontend and backend.

Vercel – Frontend

The Next.js frontend was deployed using Vercel.

Vercel was selected because it provides convenient deployment and hosting for Next.js applications.

The frontend was configured to communicate with the publicly available backend API rather than the local development server.

Render – Backend

The FastAPI backend and machine learning components were deployed using Render.

The deployed backend contains:

- FastAPI application
- CatBoost model
- Feature configuration
- Label encoder
- Required Python dependencies

Render provides a publicly accessible API endpoint that can receive prediction requests from the frontend.

Final Deployment Architecture

User

↓

Next.js Frontend

↓

Vercel

↓

FastAPI REST API

↓

Render

↓

CatBoost Model

↓

Disease Prediction

↓

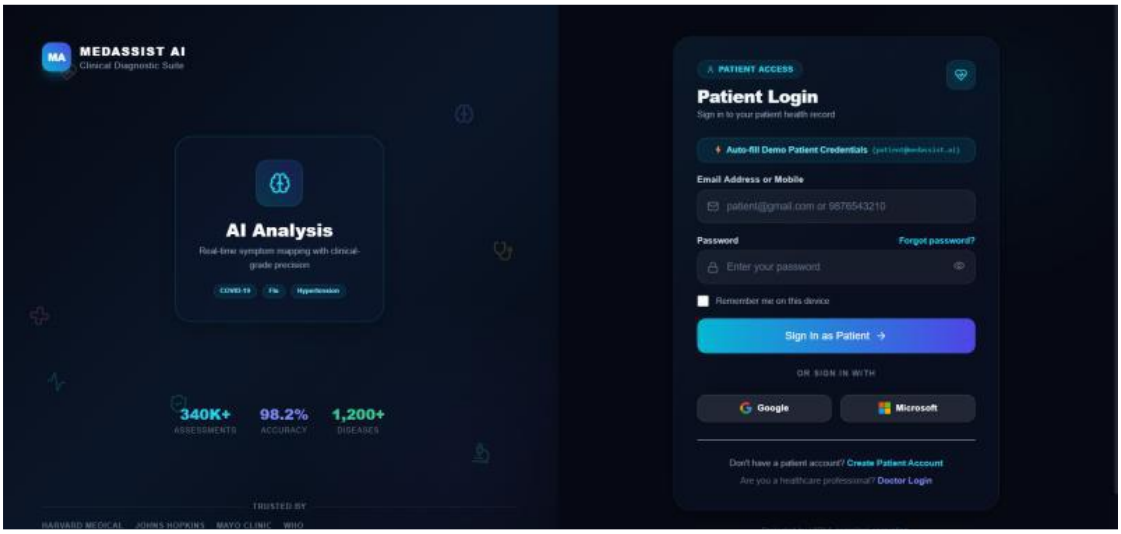
Disease Information

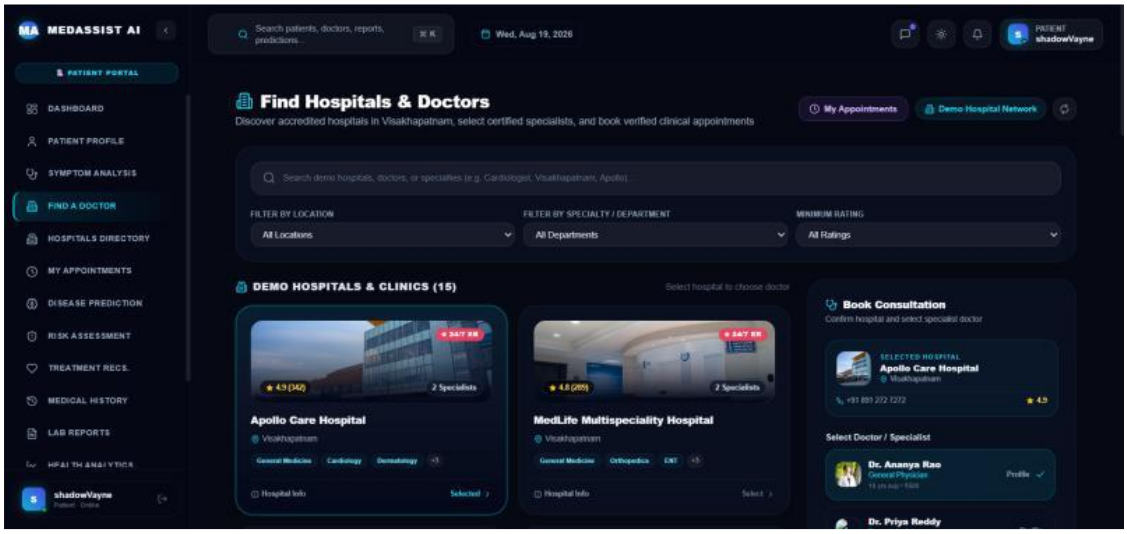
↓

Frontend Result

The final deployed application can be accessed here:

https://healthassistai-one.vercel.app/





10.My Contribution

My primary contribution to the project was focused on the **machine-learning and disease-prediction component**.

The major activities included:

Data Analysis

- Examined the large disease-symptom dataset.
- Performed EDA.
- Analyzed disease distribution.
- Checked missing values.
- Worked with the filtered 240-disease dataset.

Machine Learning

- Prepared the training and testing data.
- Trained multiple machine-learning algorithms.
- Evaluated models using accuracy, precision, recall, and F1 score.
- Compared model performance.
- Obtained approximately **86.60% accuracy using Naive Bayes**.
- Collaborated with the team on selecting CatBoost as the final integrated model.

Model Integration

- Worked with the saved CatBoost model.
- Verified the model and feature configuration.
- Verified the 377 feature columns and 240 disease classes.
- Developed and tested the disease prediction pipeline.

Disease Information

- Worked on connecting the predicted disease to a separate disease-information dataset.
- Designed the lookup structure for description, precautions, diet, lifestyle, specialist and severity information.

Application Integration

- Supported integration of the machine-learning model with the backend.
- Tested the prediction API workflow.
- Participated in final application integration and deployment preparation.

11.Conclusion

The MedAssist AI project successfully progressed through the complete machine-learning application development lifecycle, beginning with dataset exploration and model development and concluding with frontend-backend integration and cloud deployment.

A large disease-symptom dataset containing **151,556 records and 377 symptom features** was analyzed and filtered to a set of **240 diseases**. Multiple machine-learning algorithms were trained and evaluated using accuracy, precision, recall, and F1 score. The experiments demonstrated that several models were capable of achieving approximately 80–87% classification accuracy, with Naive Bayes achieving **86.60% accuracy** in the individual model experiments and the team's CatBoost model achieving approximately **86.30% accuracy**.

Based on the overall project evaluation and integration requirements, **CatBoost was selected as the final disease prediction model**.

The trained model was then verified with its corresponding feature configuration and label encoder to ensure consistency between training and inference. A reusable symptom-based prediction pipeline was developed to convert user-selected symptoms into the 377-feature representation required by the model and produce the corresponding disease prediction.

To make the prediction more useful to users, the predicted disease was connected to a separate disease-information dataset containing descriptions, precautions, medicine-related information, diet recommendations, lifestyle recommendations, specialist information, and severity.

The machine-learning component was integrated with a **FastAPI backend**, while a **Next.js frontend** was developed to provide an interactive user interface. The frontend and backend communicate through REST APIs, allowing user symptoms to flow through the complete prediction pipeline.

Finally, the frontend was deployed using **Vercel**, while the FastAPI backend and machine-learning model were deployed using **Render**. End-to-end testing confirmed that users could submit symptoms through the deployed application, communicate with the backend, obtain predictions from the CatBoost model, retrieve associated disease information, and view the results through the frontend.

Therefore, the project successfully achieved its primary objective of developing a **complete AI-based medical symptom analysis and disease prediction application**, covering data analysis, machine learning, model integration, backend development, frontend development, deployment, and testing.